

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Бинарное дерево поиска»

Выполнили:
студенты групп 24ВВВЗ

Савин А. В.

Майер В. С.

Приняли:

Юрова О. В.

Деев М. В.

Пенза 2025

Цель работы

Освоить на практике работу с бинарным деревом.

Лабораторное задание

1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.
3. * Изменить функцию добавления элементов для исключения добавления одинаковых символов.
4. * Оценить сложность процедуры поиска по значению в бинарном дереве.

Работа программы

```
-1 - окончание построения дерева
Введите число: 10
Введите число: 20
Введите число: 30
Введите число: 40
Введите число: -1
Построение дерева окончено

Дерево:
10
  20
    30
      40

Введите значение для поиска: 30
элемент 30 под номером 3 находится в дереве
```

Рисунок 1 - вывод числа из дерева

```

Дерево:
  4
10      20
  20      20
    20      30
      30      40

Введите значение для поиска: 20
элемент 20 под номером 2 находится в дереве
Введите значение для подсчета: 20
Значение 20 встречается в дереве 4 раз

```

Рисунок 2 - вывод числа вхождений числа

Вывод

Освоили на практике работу с бинарным деревом.

Листинг

```

#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <conio.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* root = NULL;

int search_tree(struct Node* r, int value, int* count) {
    if (r == NULL) {
        return 0;
    }

    (*count)++;

    if (r->data == value) {
        printf("элемент %d под номером %d находится в дереве\n", value, *count);
        return 1;
    }

    if (value < r->data) {
        return search_tree(r->left, value, count);
    }
    else {
        return search_tree(r->right, value, count);
    }
}

int count_occurrences(struct Node* r, int value) {
    if (r == NULL) {

```

```

        return 0;
    }

    int count = 0;
    if (r->data == value) {
        count = 1;
    }

    return count + count_occurrences(r->left, value) + count_occurrences(r->right, value);
}

struct Node* CreateTree(struct Node* r, int data) {
    if (r == NULL) {
        r = (struct Node*)malloc(sizeof(struct Node));
        r->left = NULL;
        r->right = NULL;
        r->data = data;
        return r;
    }

    if (data < r->data) {
        r->left = CreateTree(r->left, data);
    }
    else {
        r->right = CreateTree(r->right, data);
    }

    return r;
}

void print_tree(struct Node* r, int l) {
    if (r == NULL) {
        return;
    }

    print_tree(r->left, l + 1);

    for (int i = 0; i < l; i++) {
        printf("  ");
    }

    printf("%d\n", r->data);

    print_tree(r->right, l + 1);
}

void free_tree(struct Node* r) {
    if (r == NULL) {
        return;
    }
    free_tree(r->left);
    free_tree(r->right);
    free(r);
}

int main() {
    setlocale(LC_ALL, "");
    int D, start = 1;
    int search_value;

    root = NULL;
    printf("-1 - окончание построения дерева\n");

    while (start) {
        printf("Введите число: ");
    }
}

```

```

        scanf("%d", &D);
        if (D == -1) {
            printf("Построение дерева окончено\n\n");
            start = 0;
        }
        else {
            root = CreateTree(root, D);
        }
    }

    printf("Дерево (вертикальный вывод):\n");
    print_tree(root, 0);
    printf("\n");

    int c;
    while ((c = getchar()) != '\n' && c != EOF);

    printf("Введите значение для поиска: ");
    scanf("%d", &search_value);

    int pos = 0;
    int found = search_tree(root, search_value, &pos);

    if (!found) {
        printf("Значение %d не найдено в дереве.\n", search_value);
    }

    while ((c = getchar()) != '\n' && c != EOF);

    printf("Введите значение для подсчета: ");
    scanf("%d", &search_value);

    int occurrences = count_occurrences(root, search_value);
    printf("Значение %d встречается в дереве %d раз\n", search_value, occurrences);

    free_tree(root);

    printf("Нажмите любую клавишу для выхода...");
    _getch();
    return 0;
}

```